SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
॥वसुधैव कुटुम्बकम्॥ Re-Accredited by NAAC with 'A++' Grade

CSE - Academic Year 2026-27
Java Programming Lab | Batch 2025-29

ASSIGNMENT : 16

Name of Student	Anushree Shinde
PRN No.	25070122041
Batch	CSE A - 2
Class	Java Programming Lab
Academic Year & Semester	2025-29 : Semester 3
Date of Submission	

TITLE : Write a Java program to create and handle a user-defined exception.

Learning Objectives : To create and implement user-defined exceptions for handling application-specific errors in Java.

Theory : User-defined exceptions are exceptions created by the programmer to handle application-specific errors. In Java, a custom exception is created by extending the Exception class. These exceptions allow developers to define meaningful error messages based on specific business rules or validation requirements.

Custom exceptions improve program readability and simplify debugging by clearly identifying the cause of an error. They are useful when predefined Java exceptions do not adequately represent a particular application problem. User-defined exceptions are commonly used in enterprise software, banking systems, validation systems, and other applications that require customized error handling.

Exercise :

1. Create a Voting System that throws a custom exception if age is below 18 import java.util.Scanner.

java16a.java > Language Support for Java(TM) by Red Hat > java16a > main(String[])

```java
import java.util.Scanner;

class InvalidAgeException extends Exception {

    InvalidAgeException(String message) {
        super(message);
    }
}

public class java16a {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print(s: "Enter your age: ");
        int age = sc.nextInt();
        sc.close();

        try {
            if (age < 18) {
                throw new InvalidAgeException(message: "You are not eligible to vote.");
            }

            System.out.println(x: "You are eligible to vote.");

        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

OUTPUT :

```
Enter your age: 17
You are not eligible to vote.
```

```
Enter your age: 18
You are eligible to vote.
```

\

2. Create a Driving License System that throws a custom exception if the user's age is below 18. If the age is valid, display that the user is eligible for a driving license

java16b.java > Java > java16b

```java
import java.util.Scanner;

class InvalidAgeException extends Exception {

    InvalidAgeException(String message) {
        super(message);
    }
}

public class java16b{
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print(s: "Enter your age: ");
        int age = sc.nextInt();
        sc.close();
        try {
            if (age < 18) {
                throw new InvalidAgeException(
                    message: "You are not eligible for a driving license."
                );
            }

            System.out.println(
                x: "You are eligible for a driving license."
            );

        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

OUTPUT :

```
Enter your age: 18
You are eligible for a driving license.
PS C:\Users\softlink\Desktop\java code>
```

```
Enter your age: 21
You are eligible for a driving license.
```

Outcome :

- Understood the concept of **user-defined exceptions** in Java.
- Learned to create custom exception classes by extending the Exception class.
- Implemented custom exceptions for application-specific validation.

Conclusion :

This assignment demonstrated how to create and handle **user-defined exceptions** in Java. Custom exceptions provide meaningful error messages for specific application requirements, improve code readability, simplify debugging, and make error handling more structured and effective.